Bitácora técnica

Sistema de corrección de ejercicio mediante visión por computador

Proyectos	Proyecto-de-pose (Python) · Camcel (Flutter)
Tecnologías	MediaPipe, OpenCV, CustomTkinter, Flutter, Dart, Kotlin
Fecha	13/09/2026
Estado	Fase 2 completada · Fase 3 en preparación

1. Resumen del trabajo

El objetivo general es una aplicación móvil que analice ejercicios corporales (sentadilla, plancha) en tiempo real con la cámara, detecte errores de ejecución y entregue retroalimentación correctiva. La estrategia acordada es prototipar toda la lógica en Python sobre PC —donde iterar y depurar es rápido— y migrar a móvil al final.

Durante esta sesión se avanzó en dos frentes paralelos que terminaron conectados entre sí:

- **Proyecto-de-pose** (Python): detección de pose, cálculo de ángulos articulares, indicadores de postura de espalda y una interfaz de escritorio para pruebas.
- **Camcel** (Flutter): aplicación Android propia que transmite la cámara del celular por WiFi, reemplazando la app de terceros IP Webcam y sirviendo como primer contacto real con desarrollo móvil.

La conexión entre ambos es el protocolo MJPEG: Camcel expone el video en `/video` con el mismo formato que IP Webcam, de modo que los scripts de Python funcionan sin modificación alguna apuntando a cualquiera de los dos.

2. Proyecto-de-pose (Python)

2.1 Entorno y modelos

Se verificó el entorno virtual con OpenCV 5.0.0, MediaPipe 1.0.1 y NumPy 2.5.3, y se descargaron los modelos preentrenados oficiales de Google:

- `pose_landmarker_lite.task` (5.7 MB) — 33 landmarks corporales.
- `hand_landmarker.task` (7.8 MB) — 21 landmarks por mano.

Un archivo `.task` es una red neuronal ya entrenada empaquetada en binario. No es código: el script la carga y la usa para inferencia. Por cada frame, el modelo localiza a la persona y predice las coordenadas de sus articulaciones, normalizadas entre 0 y 1 respecto al ancho y alto de la imagen.

2.2 Ángulos articulares (Fase 2)

Se implementó `calcular_angulo(a, b, c)`, que devuelve el ángulo en el vértice `b` usando `atan2` sobre cada vector y normalizando el resultado al rango 0–180°. Con ella se calculan el ángulo de rodilla (profundidad de la sentadilla) y el de cadera (inclinación del tronco).

Nota conceptual: se descartó explícitamente el uso de Denavit-Hartenberg. DH sirve para propagar transformaciones a lo largo de una cadena cinemática de estructura conocida; aquí MediaPipe ya entrega las posiciones absolutas de cada articulación, así que el problema se resuelve con geometría vectorial directa.

2.3 Postura de espalda: limitación y solución

Problemática. Se requería evaluar si la espalda está recta o encorvada, pero MediaPipe Pose entrega siempre los mismos 33 landmarks —en cualquiera de sus tres tamaños (lite, full, heavy)— y ninguno se ubica entre hombros y caderas. No existe un punto intermedio de la columna que leer. Los modelos que sí reconstruyen la curvatura real (mallas 3D tipo SMPL) son de investigación, pesados y no ejecutables en tiempo real en un celular.

Solución. Inferir la postura por geometría sobre los puntos disponibles, mediante dos indicadores derivados:

- **Ángulo de inclinación del tronco:** se traza el eje entre el punto medio de caderas y el punto medio de hombros, y se mide su desviación respecto a la vertical. 0° equivale a tronco erguido.
- **Índice de encorvamiento:** cuantifica cuánto se adelanta la cabeza respecto al tronco, normalizado por la longitud del torso para que sea independiente de la distancia a la cámara.

2.4 Falso positivo de encorvamiento (corrección importante)

Problemática detectada en pruebas. Al agacharse a recoger un objeto con la espalda perfectamente recta, el sistema reportaba «posible encorvamiento». La primera implementación medía el desplazamiento *horizontal* de la oreja respecto a la línea de hombros; al inclinarse todo el cuerpo, tronco y cabeza rotan juntos y ese desplazamiento crece aunque no haya curvatura alguna en la espalda.

Solución. Se cambió la métrica por la distancia *perpendicular al propio eje del tronco* (producto cruz en 2D entre el vector cadera→hombro y el vector cadera→oreja, normalizado). Al medir respecto al tronco y no respecto al horizonte de la imagen, el indicador se vuelve invariante a la rotación global del cuerpo: si tronco y cabeza giran juntos, la oreja permanece sobre la línea y el índice se mantiene bajo. Solo sube cuando la cabeza se adelanta *relativa al tronco*, que es el encorvamiento real.

2.5 Interfaz de escritorio

Se construyó `app_pose.py` con CustomTkinter, aplicando una separación explícita entre cálculo y visualización: el panel de video muestra únicamente el esqueleto, mientras los valores numéricos (ángulos, inclinación, índice de encorvamiento, FPS, clasificación de postura) se mantienen como variables de estado y se presentan en un panel lateral.

Problema	Causa	Solución aplicada
El video se mostraba diminuto dentro de un área grande y vacía.	Se escalaba a un alto fijo en píxeles, sin considerar el espacio real disponible del panel.	Se escucha el evento `<Configure>` del contenedor y se escala el frame al espacio disponible conservando la relación de aspecto, sea vertical u horizontal.
Había que reescribir la IP completa del celular en cada sesión.	Un único campo de texto libre para toda la URL.	Selector «PC / Celular»: en modo celular solo se edita el último número de la IP; el prefijo y el puerto quedan fijos.
Textos del panel aparecían recortados al maximizar la ventana.	El panel lateral se estiraba proporcionalmente con la ventana.	Ancho fijo de 340 px para el panel; todo el espacio sobrante se asigna al video.
Las líneas verdes del esqueleto se veían muy gruesas.	Grosor fijo en píxeles dibujado sobre un frame de baja resolución que luego se amplía.	Grosor y radio proporcionales a las dimensiones del frame.

2.6 Rotación de persona activa mediante aplauso

Se creó `deteccion_aplausos.py`, que detecta hasta cuatro personas simultáneamente y ejecuta en paralelo el detector de manos. Cuando dos manos se aproximan por debajo de un umbral de distancia, se rota la persona «activa» (la analizada), resaltada en amarillo. Incluye un tiempo de espera de 1,5 s para no rotar varias veces con un mismo aplauso.

Limitación documentada: MediaPipe Pose no asigna identificadores estables por persona entre frames, de modo que el orden de la lista puede alterarse si los sujetos se cruzan. Es aceptable para el prototipo; un uso robusto requeriría re-identificación.

3. Camcel: aplicación Flutter propia

IP Webcam resolvía la transmisión de video, pero era una dependencia de terceros y no permitía controlar latencia ni formato. Se desarrolló Camcel como reemplazo propio, aprovechando además que el destino final del proyecto es una app móvil.

3.1 Montaje del entorno

- Flutter SDK 3.47.4 instalado en `C:\dev\flutter` y agregado al PATH.
- Android Studio con SDK 36, build-tools y platform-tools; licencias aceptadas por línea de comandos.
- `JAVA_HOME` apuntando al JBR incluido en Android Studio.
- Pruebas sobre dispositivo físico (Poco F5) vía depuración USB.

3.2 Arquitectura

La aplicación levanta un servidor HTTP dentro del propio celular que emite un stream MJPEG. Un MJPEG es una respuesta HTTP que nunca termina: en lugar de un archivo, envía imágenes JPEG consecutivas separadas por un marcador. Es exactamente el formato que `cv2.VideoCapture` ya sabe consumir.

Archivo	Responsabilidad
`main.dart`	Interfaz: preview, selector de resolución, IP mostrada, botón de transmisión, FPS.
`mjpeg_server.dart`	Servidor HTTP y emisión del stream MJPEG con descarte de frames.
`conversor_nativo.dart`	Puente (method channel) hacia el compresor JPEG nativo de Android.
`conversor_isolate.dart`	Isolate persistente; respaldo si la ruta nativa no está disponible.
`conversor_frame.dart`	Conversión YUV420 a JPEG en Dart puro (respaldo).
`MainActivity.kt`	Código Kotlin que comprime NV21 a JPEG con `YuvImage`.

3.3 Problemas de compilación e instalación

Problema	Causa	Solución aplicada
`INSTALL_FAILED_USER_RESTRICTED` al instalar el APK.	MIUI exige la opción «Instalar vía USB» y muestra un diálogo de confirmación en pantalla.	Activar la opción en Opciones de desarrollador, mantener el teléfono desbloqueado y usar `adb install -r` directamente.
La app instalada seguía mostrando el contador de ejemplo.	`flutter install` instala el APK existente pero no lo recompila.	Ejecutar `flutter build apk` antes de instalar.
`AccessDeniedException` y «not a regular file» en la carpeta build.	OneDrive sincronizaba los artefactos de compilación mientras Gradle los escribía, sustituyéndolos por marcadores de nube.	Se trasladó el proyecto a `C:\dev\camcel`, fuera de la carpeta sincronizada. Gradle no es compatible con carpetas en sincronización activa.
`CallbackToFutureAdapter not found` al compilar el plugin de cámara.	Dependencia transitiva ausente en la versión del plugin `camera_android_camerax` instalada.	Actualización del paquete `camera` a la versión 0.12.1, donde el problema está corregido.

3.4 Rendimiento: de entrecortado a fluido

La primera versión funcionaba pero el video se veía a tirones. El diagnóstico fue que el trabajo pesado —convertir cada frame de YUV420 a RGB y luego codificarlo a JPEG— se ejecutaba píxel a píxel en Dart, sobre el mismo isolate que dibuja la interfaz. En un Poco F5 (Snapdragon 7+ Gen 2, ocho núcleos) se estaba usando un solo núcleo.

La optimización se hizo en tres etapas acumulativas:

- **Aritmética de punto fijo.** Se reemplazaron las operaciones en coma flotante por enteros con desplazamientos de bits, y se escribió directamente sobre un búfer de bytes en lugar de invocar `setPixelRgb` por píxel.
- **Isolate dedicado.** La conversión se movió a un isolate persistente —el equivalente a un hilo en Dart, sin memoria compartida— para dejar de bloquear la interfaz y el preview de cámara.
- **Codificación nativa.** Se implementó un *method channel* hacia Kotlin que usa `YuvImage.compressToJpeg()` de Android, código nativo optimizado. Además se solicitó a la cámara el formato NV21 (búfer único), eliminando por completo el paso intermedio a RGB, que era redundante: JPEG ya trabaja internamente en YCbCr.

Se añadieron también un selector de resolución (240p / 480p / 720p) y un contador de FPS en pantalla, de modo que el compromiso entre calidad y fluidez se pueda medir en lugar de suponerse.

3.5 Latencia acumulada de un minuto

Problemática. Aun con el video fluido, lo que mostraba Python iba cerca de un minuto por detrás de la realidad, y el retraso crecía con el tiempo. Ese comportamiento acumulativo es la firma de un búfer que se llena más rápido de lo que se vacía.

Causas, en ambos extremos:

- En el PC, `cv2.VideoCapture` encola internamente los frames que llegan por red. Como MediaPipe tarda más en procesar un frame de lo que tarda en llegar el siguiente, la cola crecía indefinidamente.
- En el celular, el servidor escribía cada frame al socket sin verificar si el anterior ya había salido, acumulándolos en el búfer de escritura.

Solución. Aplicar el mismo principio en los dos lados: *para video en vivo, es preferible perder frames que atrasarse*. En el celular, si el cliente no terminó de recibir el frame anterior, el nuevo se omite. En el PC se añadió la clase `LectorCamara`, que lee en un hilo aparte y conserva únicamente el frame más reciente, descartando los anteriores.

4. Estado actual y próximos pasos

Completado

- Detección de pose con ángulos de rodilla y cadera.
- Indicadores de postura de espalda, corregidos contra falsos positivos.
- Interfaz de escritorio con separación entre cálculo y visualización.
- Detección de aplauso para rotar entre personas.
- App Android propia transmitiendo video con baja latencia.

Pendiente inmediato

- **Recolectar datos reales** para calibrar umbrales: rango de ángulo de rodilla en sentadilla completa, y valores de inclinación e índice de encorvamiento en postura recta frente a encorvada. Los umbrales actuales son provisionales y deben sustituirse por mediciones propias.
- **Fase 3:** máquina de estados para contar repeticiones (de pie → bajando → abajo → subiendo → de pie) y detección de errores comunes como valgo de rodilla o talones despegados.
- **Decisión de diseño abierta:** definir el ángulo de captura. La evaluación de espalda funciona mejor de perfil; la de rodillas, de frente.
- **Fase 4 en adelante:** retroalimentación al usuario y migración de la lógica a Flutter.